

Module 5

RAG avancé & évaluation

Cursus AI Engineer · Neosoft · 1 jour

Positionnement du module

Vous avez désormais (M4) :

- Pipeline de recherche vectorielle production-ready
- Hybride dense + sparse + RRF
- Métriques de retrieval (recall, MRR)

Ce module construit la **couche RAG complète** au-dessus :

- Techniques de transformation de requête
- Re-ranking par cross-encoder
- Compression contextuelle
- **Évaluation Ragas** : faithfulness, relevancy, precision, recall
- Intégration en CI/CD

*Sans évaluation rigoureuse, **vous optimisez à l'aveugle**. C'est le sujet du module.*

Objectifs pédagogiques

À l'issue du module, vous serez capable de :

1. **Diagnostiquer** un naïve RAG et identifier les patterns à appliquer
2. **Implémenter** query rewriting, multi-query, HyDE
3. **Intégrer** un re-ranker cross-encoder dans le pipeline
4. **Versionner** prompts et configurations RAG
5. **Évaluer** un RAG avec Ragas (4 métriques clés)
6. **Construire** un golden set défendable
7. **Bloquer** une régression via la CI

Plan de la journée

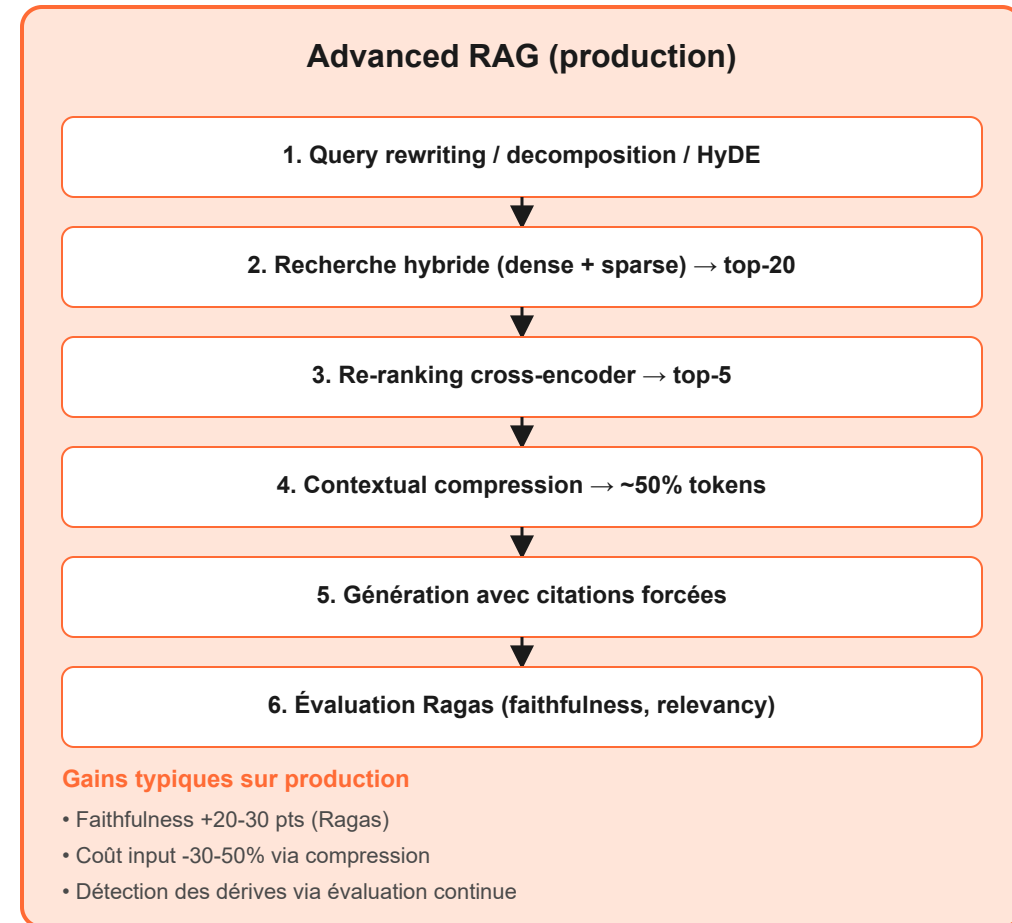
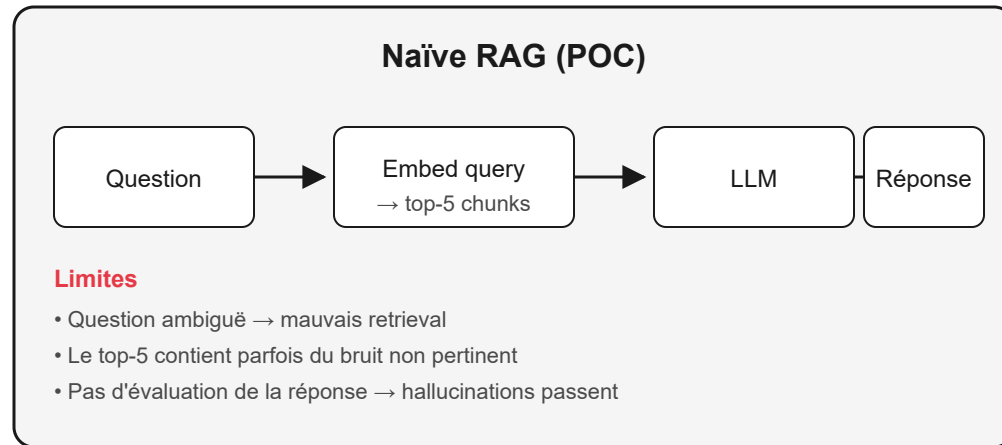
Bloc	Contenu	Durée
Matin	Pourquoi le naïve RAG ne suffit pas	30 min
	Query rewriting, multi-query, HyDE	1h
	Re-ranking cross-encoder	1h
	Contextual compression et long-context	30 min
Après-midi	Évaluation Ragas : 4 métriques	1h
	Construire un golden set	30 min
	Versioning prompts (MLflow Prompt Registry)	30 min
	TPs : pipeline complet + CI	1h30

Partie 1

Du naïve RAG à la production

Naïve RAG vs Advanced RAG

Naïve RAG vs Advanced RAG



Pourquoi le naïve RAG échoue en production

5 modes de défaillance fréquents :

1. **Question ambiguë** : « comment je le configure ? » — référent flou → mauvais retrieval
2. **Vocabulaire utilisateur ≠ vocabulaire docs** : « accès cassé » vs « erreur d'authentification »
3. **Bruit dans le top-k** : 5 chunks récupérés, 2 hors-sujet → le LLM s'égare
4. **Hallucinations subtiles** : le LLM extrapole au-delà du contexte
5. **Pas de monitoring** : la qualité dérive en silence

Aucun de ces problèmes n'est visible avec une démo de 5 questions choisies. D'où l'évaluation systématique.

La règle de production

******You never put a RAG in prod without : (1) golden set, (2) Ragas metrics, (3) gate CI.******

Ces 3 éléments ne sont **pas** une option. Ils sont la condition de pouvoir améliorer le système plus tard.

Sans eux, chaque modification du prompt, du chunking, du modèle est un **pari aveugle**.

Partie 2

Transformer la requête

Les 4 techniques principales

Techniques de transformation de requête

Question originale : « j'arrive pas à me connecter, j'ai changé de portable la semaine dernière »

Query Rewriting

Reformuler en requête de recherche claire

→ « **Comment réinitialiser l'authentification multi-facteurs après changement de mobile ?** »

- + Capture l'intention réelle
- + Termes techniques pour BM25
- 1 appel LLM en plus (~50 tokens)

Sub-question decomposition

Décomposer en sous-questions atomiques

→ **Q1 : « Réinitialiser MFA NovaCloud »**
→ **Q2 : « Procédure perte de smartphone MFA »**

- + Améliore le recall sur questions complexes
- Coût × N (1 retrieval par sous-question)
- Fusion des résultats à gérer

HyDE (Hypothetical Doc Embed)

Générer une réponse hypothétique, l'embedder

LLM → « Pour réinitialiser le MFA après un changement de mobile, allez dans Paramètres > ... »

- + Distribution sémantique proche des docs
- Risque d'hallucination si modèle faible
- 1 appel LLM en plus (200-300 tokens)

Multi-query expansion

Générer 3-5 paraphrases, fusionner les résultats

→ « Reset MFA après nouveau téléphone »
→ « Récupérer accès après perte authenticator »
→ « Désactiver 2FA pour reconfigurer »

- + Robuste au vocabulaire
- Coût × N requêtes, fusion RRF nécessaire

Query Rewriting — implémentation

```
REWRITE_PROMPT = """Tu es un assistant qui reformule des questions utilisateurs  
en requêtes de recherche claires et complètes.
```

```
Question utilisateur : {question}
```

```
Reformule en une question explicite incluant :
```

- L'intention claire
- Les termes techniques pertinents si pertinent
- Le contexte implicite (produit, version) s'il peut être déduit

```
Réponds UNIQUEMENT avec la question reformulée, sans préambule."""
```

```
def rewrite_query(question: str, llm) -> str:  
    return llm(REWRITE_PROMPT.format(question=question)).strip()
```

```
# Usage
```

```
original = "j'arrive pas à me connecter, j'ai changé de portable la semaine dernière"  
rewritten = rewrite_query(original, llm)
```

Coût : 1 appel LLM léger (Haiku, GPT-4o-mini). Latence ~300-500 ms. Worth it.

Multi-query expansion

Générer **N paraphrases** et fusionner les résultats avec RRF :

```
def multi_query_search(question: str, llm, retriever, n: int = 4) -> list:
    # 1. Générer N paraphrases
    prompt = f"""Génère {n} reformulations DIFFÉRENTES de cette question, une par ligne :
    Question : {question}"""
    paraphrases = [question] + llm(prompt).strip().split("\n")[:n]

    # 2. Récupérer pour chacune
    rankings = [retriever.search(p, top_k=20) for p in paraphrases]

    # 3. Fusionner avec RRF
```

Quand c'est rentable :

- Questions courtes ambiguës
- Vocabulaire utilisateur très varié
- Domaines sans terminologie stable

Coût × N retrievals + 1 appel LLM. Mesurer avant d'activer en prod.

HyDE — Hypothetical Document Embeddings

Générer une **fausse réponse** et embedder cette réponse plutôt que la question :

```
HYDE_PROMPT = """Réponds brièvement (3-4 phrases) à cette question, comme si  
tu connaissais déjà la réponse. Sois factuel et utilise un ton documentation.
```

```
Question : {question}
```

```
Réponse :"""
```

```
def hyde_search(question: str, llm, embedder, retriever):  
    hypothetical = llm(HYDE_PROMPT.format(question=question))  
    # Embedder la fausse réponse, pas la question  
    fake_answer_embedding = embedder.embed(hypothetical)  
    return retriever.search_by_vector(fake_answer_embedding, top_k=10)
```

Pourquoi ça marche : la fausse réponse a une distribution sémantique **proche des vrais documents**.

Risque : si le modèle hallucine la réponse, l'embedding pointe vers le vide. Mesurer.

Sub-question decomposition

Pour les questions complexes nécessitant **plusieurs informations** :

```
DECOMPOSE_PROMPT = """Décompose cette question en 2-4 sous-questions atomiques.  
Chaque sous-question doit être indépendante et factuelle.  
  
Question : {question}  
  
Sous-questions (une par ligne) :"""  
  
def decompose_search(question: str, llm, retriever):  
    sub_qs = llm(DECOMPOSE_PROMPT.format(question=question)).strip().split("\n")
```

Exemple :

```
Q : « Quelle est la différence de prix et de SLA entre les plans Pro et Enterprise ? »  
  
Q1 : « Prix du plan Pro »  
Q2 : « Prix du plan Enterprise »  
Q3 : « SLA du plan Pro »  
Q4 : « SLA du plan Enterprise »
```

Quelle technique choisir ?

Cas d'usage	Technique recommandée
Questions naturelles courtes / ambiguës	Query rewriting
Vocabulaire très variable utilisateurs	Multi-query
Corpus avec termes techniques rares	HyDE
Questions multi-facettes / comparatives	Sub-question decomposition
Questions factuelles simples	Aucune (overhead inutile)

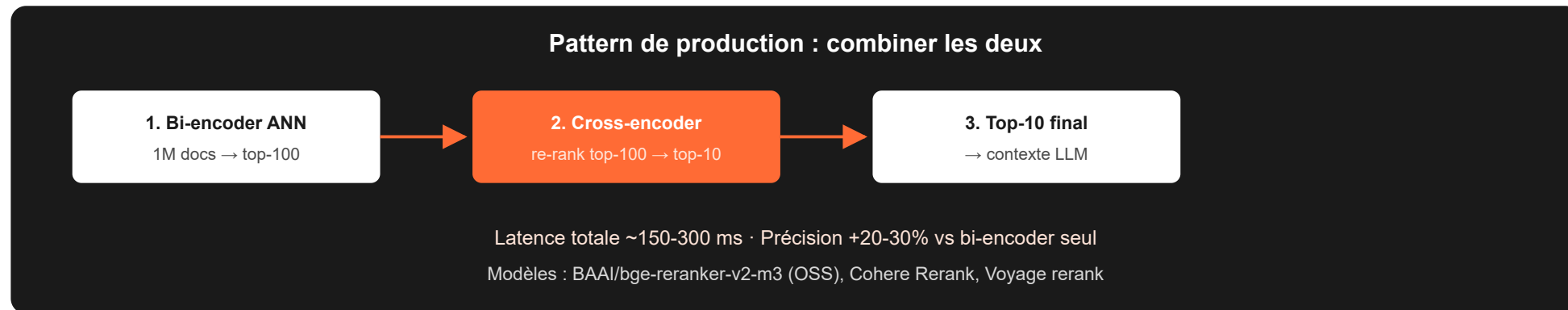
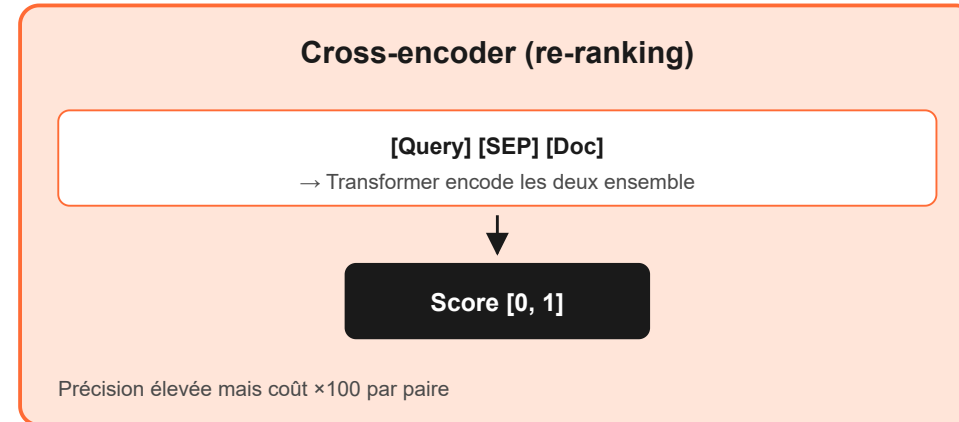
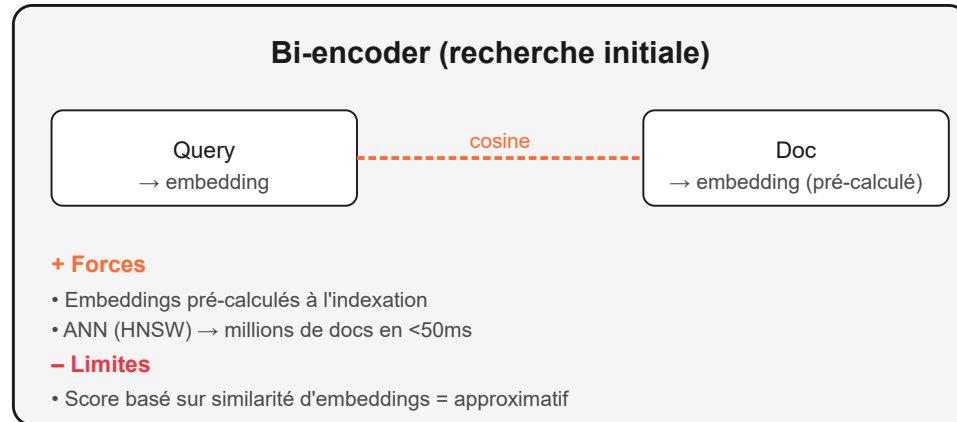
Règle pratique : ne pas activer plusieurs techniques en même temps avant d'avoir mesuré l'effet de chacune isolée.

Partie 3

Re-ranking

Bi-encoder vs Cross-encoder

Bi-encoder (retrieval) vs Cross-encoder (re-ranking)



Implémentation cross-encoder local

```
from sentence_transformers import CrossEncoder

# Modèle multilingue compact, excellent rapport qualité/coût
reranker = CrossEncoder("BAAI/bge-reranker-v2-m3", max_length=512)

def rerank(query: str, candidates: list[dict], top_k: int = 5) -> list[dict]:
    """candidates : liste de dicts avec au moins {'content': str}"""
    pairs = [(query, c["content"]) for c in candidates]
    scores = reranker.predict(pairs, show_progress_bar=False)

    # Trier par score décroissant
    ranked = sorted(zip(candidates, scores), key=lambda x: -x[1])
    return [{"chunk": c, "rerank_score": float(s)} for c, s in ranked[:top_k]]

# Usage
top_20 = vector_search(query, top_k=20)
top_5_reranked = rerank(query, top_20, top_k=5)
```

`bge-reranker-v2-m3` tourne sur CPU avec une latence acceptable jusqu'à ~50 candidats.

Le ROI du re-ranking

Sur un benchmark RAG typique (50 cas, golden set varié) :

Configuration	Faithfulness	Relevancy	Latence p95
Bi-encoder seul (top-5)	0.78	0.84	110 ms
Bi-encoder top-20 → cross-encoder top-5	0.89	0.91	280 ms
Hybride + cross-encoder	0.92	0.93	320 ms

+10-15 points de faithfulness pour ~150 ms de latence. Excellent ratio.

Partie 4

Compression contextuelle

Le problème du contexte gonflé

5 chunks × 500 tokens = **2500 tokens** envoyés au LLM.

Souvent, **20-40%** seulement sont réellement pertinents pour la question. Le reste est :

- du contexte de pré-/post-section
- des phrases connexes mais hors-sujet
- des répétitions inter-chunks

Coût direct :

- LLM input : prix × 5
- Latence : tokens à lire = TTFT plus long

Coût indirect : « lost in the middle » — le modèle peut rater l'info clé.

3 approches de compression

```
# 1. Extractive (rapide, par chunk individuel)
def extract_relevant_sentences(query: str, chunk: str, llm) -> str:
    prompt = f"""Question : {query}
    Texte : {chunk}
```

```
    Extrais UNIQUEMENT les phrases du texte qui répondent directement à la question.
    Reproduis-les MOT POUR MOT, sans paraphraser. Si rien n'est pertinent : NONE."""
    return llm(prompt)
```

```
# 2. LLMlingua / prompt compression (modèle dédié, sans LLM)
from llmlingua import PromptCompressor
compressor = PromptCompressor("microsoft/llmlingua-2-bert-base-multilingual-cased-meetingbank")
compressed = compressor.compress_prompt(chunks, target_token=800)
```

```
# 3. Re-ranker comme filtre (binaire)
# Garder uniquement les chunks dont le score de re-ranker > seuil
filtered = [c for c, score in zip(chunks, scores) if score > 0.5]
```

Quand chaque méthode brille

Méthode	Quand l'utiliser
Extractive (LLM)	Qualité maximale, mais coût × N appels LLM
LLMLingua	Compression rapide, sans LLM, jusqu'à 5x
Re-ranker comme filtre	Quasi-gratuit, à activer par défaut

*Pattern recommandé : **re-ranker filtre** + **compression LLM optionnelle** sur les chunks restants.*

Long-context : le RAG est-il mort ?

Avec Gemini 1.5 / Claude (200k+ tokens), on pourrait coller un livre entier. **Mais :**

- ✗ **Coût** : $100k \text{ input tokens} \times 10 \text{ questions} \times 1000 \text{ users/jour} = \text{explosion}$
- ✗ **Latence** : TTFT de 2-5s sur long contexte
- ✗ **« Lost in the middle »** : les modèles oublient le centre de longs contextes
- ✗ **Pas de citations** : impossible de tracer d'où vient l'info

✓ **Cas valides du long-context :**

- Analyse d'un seul gros document (rapport, contrat, livre)
- Synthèse multi-documents quand TOUT est nécessaire
- Tâches qui nécessitent une cohérence globale

*Pour la majorité des cas RAG métier : **bon retrieval** > **long contexte brut**.*

Partie 5

Évaluation Ragas

Les 4 métriques essentielles

Les 4 métriques Ragas — qu'est-ce qu'on mesure ?

CÔTÉ RETRIEVAL

Context Precision

« Les chunks récupérés sont-ils tous pertinents ? »



Faible = trop de chunks non pertinents dans le contexte (bruit)

Context Recall

« Toutes les infos nécessaires sont-elles dans le contexte ? »

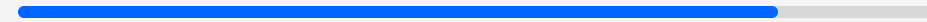


Faible = retrieval rate des docs nécessaires → réponse incomplète

CÔTÉ GÉNÉRATION

Faithfulness

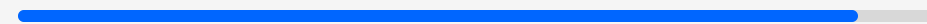
« La réponse s'appuie-t-elle uniquement sur le contexte ? »



Faible = hallucinations, faits non sourcés

Answer Relevancy

« La réponse répond-elle bien à la question ? »



Faible = réponse à côté de la plaque ou trop générale

Matrice de diagnostic — où chercher en premier ?

Tout vert (>0.85)

Système sain

Continuer le monitoring

Recall faible

Problème : retrieval
→ Augmenter top-k
→ Améliorer chunking
→ Activer hybride

Faithfulness faible

Problème : hallucinations
→ Prompt plus strict
→ Réduire temperature
→ Modèle plus fort

Relevancy faible

Problème : interprétation
→ Query rewriting
→ Few-shot examples
→ Clarifier le rôle

Faithfulness en détail

Question : la réponse contient-elle uniquement des affirmations supportées par le contexte ?

Algorithme Ragas :

1. LLM-juge décompose la réponse en **affirmations atomiques**
2. Pour chaque affirmation, vérifie si **dérivable** du contexte
3. Score = (affirmations supportées) / (total affirmations) $\in [0, 1]$

Exemple :

*Réponse : « SAML 2.0 nécessite un certificat X.509 et **est gratuit sur tous les plans** »*

Contexte : « Configurer SAML 2.0 nécessite un certificat X.509. »

2 affirmations, 1 supportée, 1 hallucinée → faithfulness = 0.5

Cible production : > 0.85. En dessous = hallucinations fréquentes.

Answer Relevancy en détail

Question : la réponse répond-elle à la question posée (et pas à côté) ?

Algorithme Ragas :

1. Pour la réponse générée, le juge LLM génère **N questions plausibles** auxquelles cette réponse correspondrait
2. Calcule la **similarité d'embeddings** entre la vraie question et les N questions générées
3. Score = moyenne des similarités $\in [0, 1]$

Exemple :

Question : « Comment activer SAML ? »

Réponse : « Pour activer SAML, allez dans Paramètres > Auth > SAML. »

Questions inverses : ["Comment activer SAML ?", "Où configurer SAML ?", ...]

Similarité moyenne $\approx 0.93 \rightarrow$ relevancy élevée

Context Precision et Context Recall

Context Precision : « Parmi les chunks récupérés, combien sont pertinents ? »

Context Recall : « Toutes les infos nécessaires sont-elles présentes ? » (nécessite ground truth)

```
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall,
)

eval_data = {
    "question": "Comment activer SAML ?",
    "answer": "Pour activer SAML, allez dans Paramètres > Auth > SAML.",
    "contexts": [doc1.text, doc2.text, doc3.text],
    "ground_truth": "Allez dans Paramètres puis Authentification puis SAML.",
}
```

*Tu n'as pas toujours de **ground truth** pour context_recall. Construire un golden set partiel : OK.*

Ragas — exemple complet

```
from datasets import Dataset
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision, context_recall
from ragas.llms import LangchainLLMWrapper
from langchain_anthropic import ChatAnthropic

# Le LLM-juge (idéalement plus fort que le LLM générateur)
judge_llm = LangchainLLMWrapper(ChatAnthropic(model="claude-sonnet-4-6"))

dataset = Dataset.from_dict({
    "question": [...],
    "answer": [...],          # généré par votre RAG
    "contexts": [[...], ...], # chunks récupérés
    "ground_truth": [...],   # réponse attendue
})

result = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy, context_precision, context_recall],
    llm=judge_llm,
)
print(result)
# {"faithfulness": 0.87, "answer_relevancy": 0.91, "context_precision": 0.83, "context_recall": 0.85}
```

Construire un golden set défendable

3 méthodes (à combiner) :

Méthode	Coût	Qualité	Volume
Annotation humaine	Élevé	★★★★★	50-100 paires
À partir de FAQ existante	Faible	★★★★	200-500 paires
Synthétique (LLM) + revue humaine partielle	Moyen	★★★	500-2000 paires

Mix recommandé :

- 50 paires gold (humaines)
- 200 paires extraites de FAQ
- 300 paires synthétiques avec **revue humaine de 10%**

Versionner le golden set en Git. Il évolue avec le produit, c'est normal.

Partie 6

Versioning et CI

MLflow Prompt Registry

Versionner les prompts comme du code :

```
import mlflow

mlflow.set_tracking_uri("http://localhost:5000")

# Enregistrer un prompt
prompt = mlflow.register_prompt(
    name="rag-system-prompt",
    template="""Tu es un expert {domain}. Réponds en t'appuyant UNIQUEMENT sur :
{context}

Question : {question}
Réponse :""",
    commit_message="v3 - ajout instruction citation",
    tags={"author": "christophe", "model": "claude-sonnet-4-6"},
)

# Charger une version spécifique
```

*Promotion dev → staging → prod via les **aliases**. Approfondi en M9.*

Boucle d'évaluation continue

Boucle d'évaluation continue en CI/CD



Exemple de gate CI GitHub Actions

```
name: rag-quality-gate
on: [pull_request]

jobs:
  evaluate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install -r requirements.txt

      - name: Run Ragas eval
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: python eval/run_ragas.py --golden tests/golden.jsonl --output results.json

      - name: Compare to baseline
        run: python eval/compare.py --new results.json --baseline main_baseline.json --threshold -0.05
```

*Job qui dure ~5 min sur 50 cas. CI échoue si **faithfulness** baisse de plus de 5 pts.*

A/B testing en production

Une fois en prod, comparer **2 versions** d'un prompt sur du trafic réel :

```
import random

def rag_pipeline(query: str, user_id: str) -> dict:
    # Bucketing déterministe
    bucket = "B" if hash(user_id) % 100 < 10 else "A" # 10% sur B
    prompt_version = "5" if bucket == "B" else "4"

    answer = generate(query, prompt_version=prompt_version)

    # Logger pour analyse a posteriori
    log_to_langfuse({
        "query": query, "answer": answer, "user_id": user_id,
        "bucket": bucket, "prompt_version": prompt_version,
    })
    return answer
```

Mesurer pendant 1-2 semaines, comparer les distributions de Ragas + thumbs up/down utilisateurs.

Partie 7

TPs

TP 1 — Pipeline RAG complet

Objectif : assembler query rewriting + hybride + re-ranking + génération.

→ Voir `exercises/exo1_rag_pipeline.md`

TP 2 — Évaluation Ragas + golden set

Objectif : construire un golden set, lancer Ragas, comparer 3 variantes de prompt.

→ Voir `exercises/exo2_ragas_eval.md`

Récapitulatif

- ✓ Diagnostiquer un naïve RAG et identifier les patterns à appliquer
- ✓ Implémenter query rewriting / multi-query / HyDE
- ✓ Intégrer un re-ranker cross-encoder
- ✓ Évaluer rigoureusement avec Ragas (4 métriques)
- ✓ Construire un golden set défendable
- ✓ Versionner les prompts et bloquer les régressions en CI

Prochain module (M6) : orchestration agentique avec LangGraph.

Questions ?

Merci pour votre attention

Support, démos et exercices : `supports/M05_rag_avance/`